Representations and Twin Networks

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

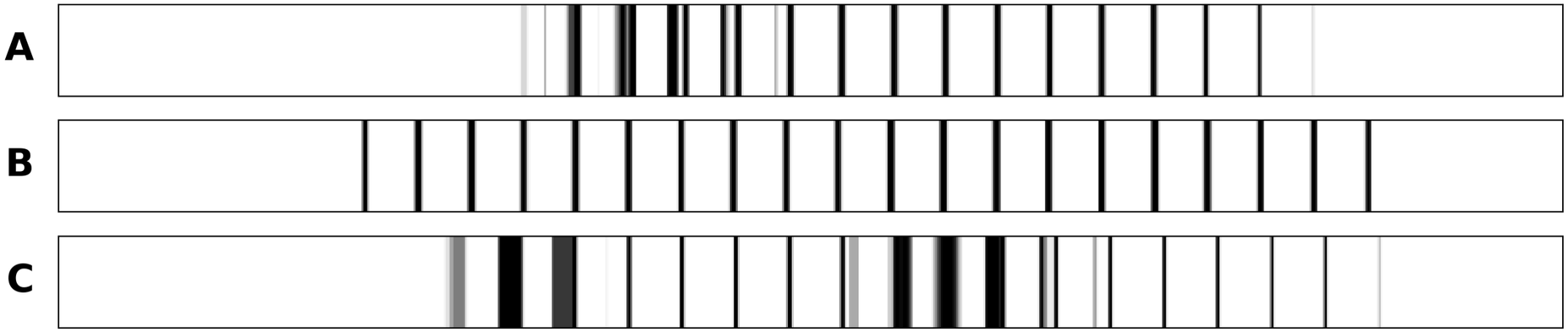
Learning objectives

- Explain what PCA computes, and read a 2-D projection of a high-dimensional representation
- Say what is different about a representation chosen by variance (PCA) and one learned for a task (a network's features)
- Reuse a trained network's features for a new problem: a new linear head, or nearest neighbors
- Explain what a twin network learns that a classifier does not, and how far that carries over to classes it never saw
- Describe a twin network (one set of weights applied to two inputs), and write the contrastive loss with its pull term, push term, and margin
- Say how a twin network is used for verification and for one-shot recognition

Same or different?

What does “the same” mean to a computer?

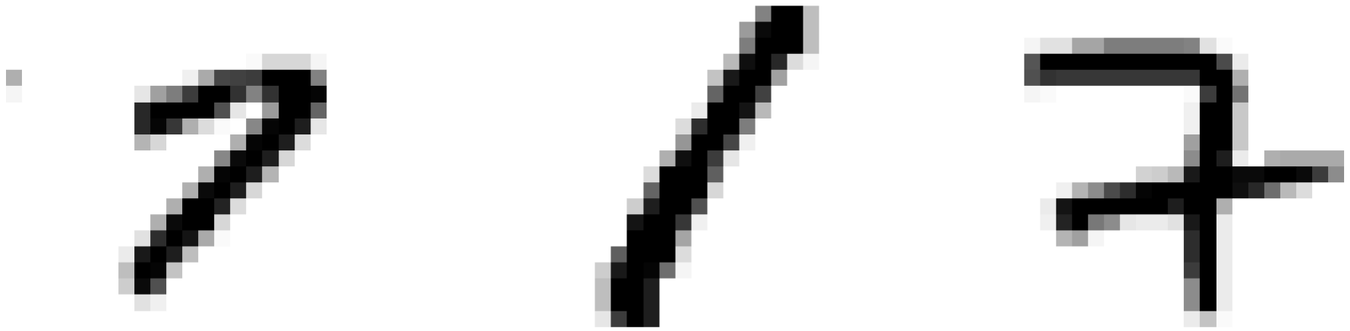
Which two are the same digit?



- Each image is 784 pixel values, laid out here as one line: row 1 of the image, then row 2, and so on
- Which two show the same digit?

Now with the shape back

A: 7 B: 1 C: 7



- Easy now: A and C are both 7s, and B is a 1
- These are the same 784 numbers. Only the layout changed
- We see a shape. A model is handed numbers, and has to be taught what “the same” means

How far apart are these vectors?

L2 distance $\sqrt{\sum_i (a_i - b_i)^2}$

smaller = more alike

	A: 7	B: 1	C: 7
A: 7	0.00	**7.12**	**8.98**
B: 1	**7.12**	0.00	**9.53**
C: 7	**8.98**	**9.53**	0.00

Cosine similarity $a \cdot b / (\|a\| \, \|b\|)$

larger = more alike

	A: 7	B: 1	C: 7
A: 7	1.00	**0.54**	**0.35**
B: 1	**0.54**	1.00	**0.30**
C: 7	**0.35**	**0.30**	1.00

The two 7s are **not** more similar to each other than to the 1

What is a representation?

A representation is a vector that describes an input.

- Any model works from some list of numbers, never from the thing itself
- The same MNIST picture can be described in many ways:
  - **Pixels:** 784 numbers, how dark each position is
  - **Hand-chosen metrics:** things like total ink, height, or “islands” of ink
  - **Learned:** a CNN’s hidden layer

Pixels are a poor representation

- A distance between pixel vectors measures how much **ink overlaps**, not whether two images show the same digit
  - Think back to when we talked about *why* CNNs a few lectures ago
  - Shift a 7 a few pixels and many pixels change, as we saw when we moved a digit around

Two representations of the same MNIST image

Pixels

- 784 numbers: how dark each position is
- Chosen by the camera placement
- Distances between them are a poor guide to “the same”

Features from a CNN

- 400 numbers: how strongly each learned pattern is present
- Chosen by **training**
- The numbers the final linear layer works from

Our Goals:

If two representations have a **high similarity** they are the **same digit**

If two representations have a **low similarity** they are **different digits**

Representations

Three steps, each about representations:

1. **PCA:** a representation chosen from the data with linear algebra, and a way to look at one
2. **Reuse:** take a representation a network already learned and use it on a new task
3. **Twin networks:** train a representation for the comparison itself, so “same or different” becomes a distance

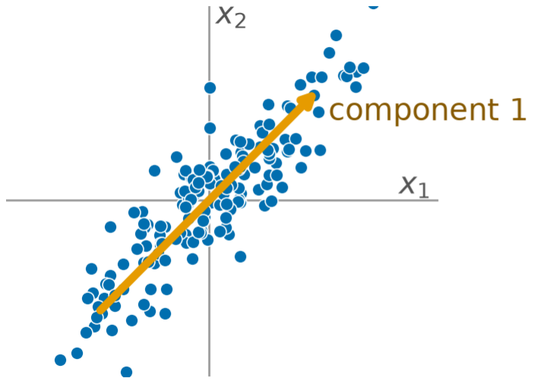
Principal component analysis (PCA)

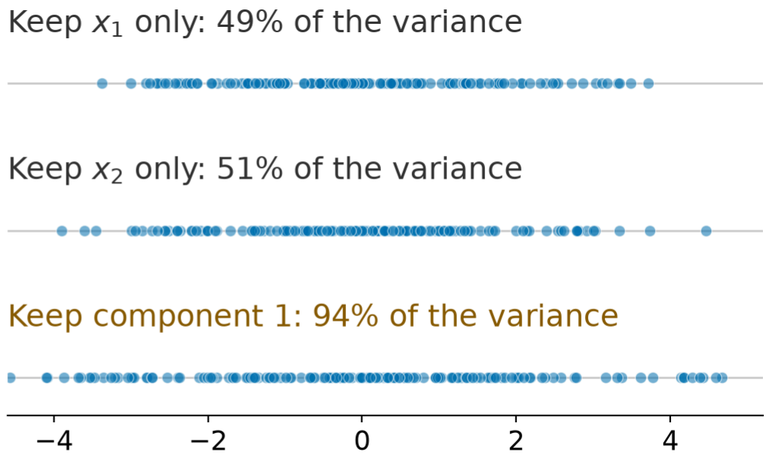
A linear representation in fewer dimensions

The Big Idea

Describe each image using only the few dimensions that keep the most variance

If we keep only one dimension

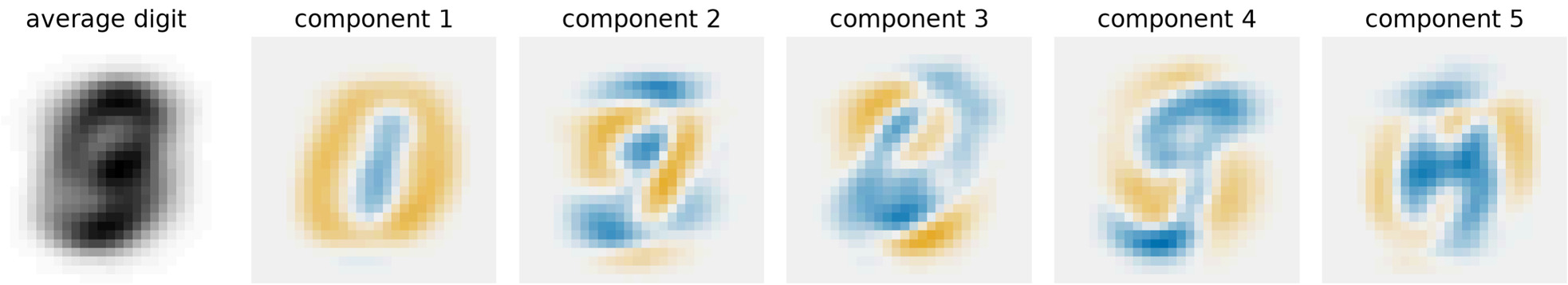




If we have to keep **only one** dimension, which one is best?

- The data has two *original* dimensions, x_1 and x_2
  - Either original dimension alone keeps about half of the **variance**: how much the points' values differ from their average
- **Component 1** is a new dimension – a linear combination of x_1 and x_2

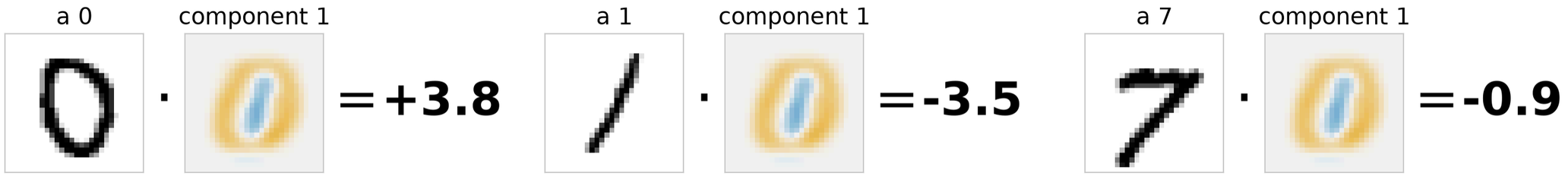
A new dimension is *just* a linear combination



- Each component is a weight w_i for every one of the 784 pixels, drawn here as a 28 × 28 image. Orange: positive weight. Blue: negative
- An image's value on a component is a weighted sum of its pixels: $z = w_1x_1 + w_2x_2 + \cdots + w_{784}x_{784}$, taken after subtracting the average digit
- 784 numbers in, one number out for each component we keep

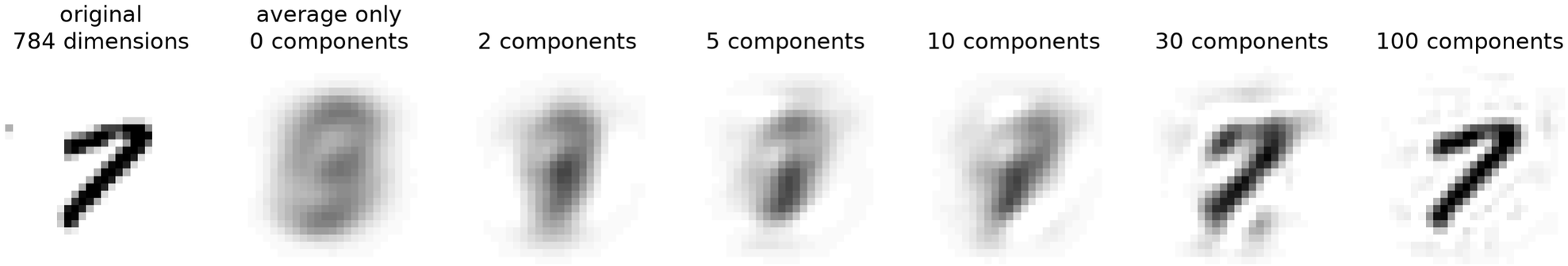
Foreshadowing: This kind of feels like filters, doesn't it?

Each value is a dot product



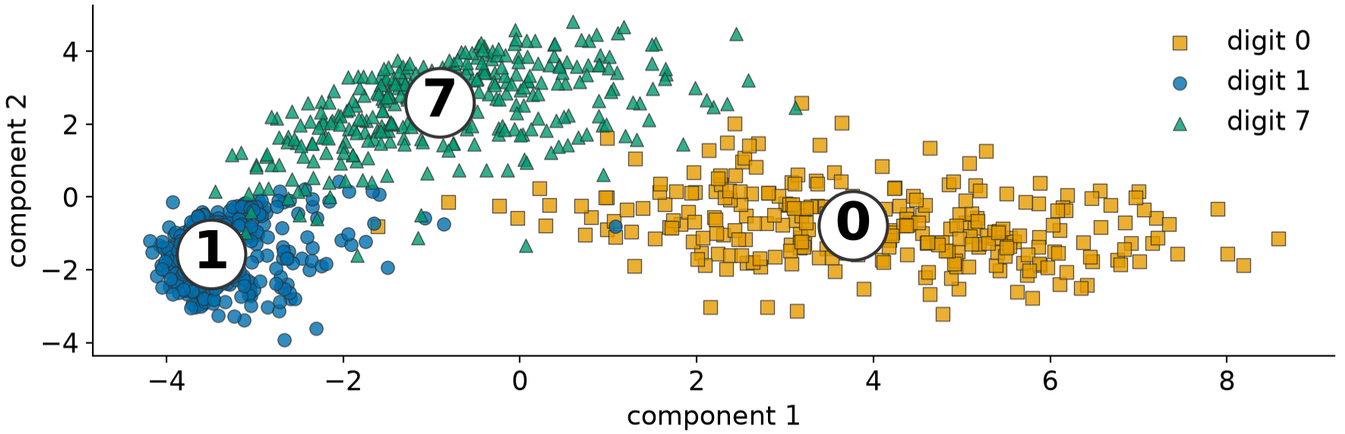
- An image's value on a component is a **dot product**: multiply each pixel by its weight, then add them all up
- Same weights every time. Only the image changes, and the value says how well the two line up
- Strictly, the average digit is subtracted from the image first. Orange weights: ink raises the value. Blue weights: ink lowers it

Rebuilding an image from a few components



- Start from the average image, then add each component's pattern in proportion to this image's value on it
- With 2 or 5 components it is a blur. By 30 it is clearly a 7, and with 100 it is close to the original
- 30 dimensions instead of 784, because PCA **picks the components** that keep as much as possible

Three digits through PCA



- Every image is now represented by a **two dimensional vector** – a linear projection of the original data
- Even in this 2 dimensional space we can see different groups
  - 1 forms a tight group
  - 0 forms a wide one
  - 7 sits above, partially overlapping the 1s

PCA: the full recipe

1. **Center the data:** Subtract the average image, $\mathbf{x} - \mu$
2. **Calculate Covariance:**
   - $C = \frac{1}{n} \sum_{j=1}^{n} (\mathbf{x}_j - \mu)(\mathbf{x}_j - \mu)^\top$
   - Finds the amount of variation in each input dimension
3. **Find Components:**
   - The eigenvectors $\mathbf{w}_1, \mathbf{w}_2, \ldots$ of C, ordered by eigenvalue $\lambda_1 \geq \lambda_2 \geq \ldots$
   - Figures out which variations are the most impactful
4. **Project onto a lower number of dimensions (e.g. k dimensions):**
   - keep the first k, $W = [\mathbf{w}_1 \ \ldots \ \mathbf{w}_k]$, and compute $\mathbf{z} = W^\top (\mathbf{x} - \mu)$

$\mathbf{z}$ is the new, k-dimensional representation of $\mathbf{x}$

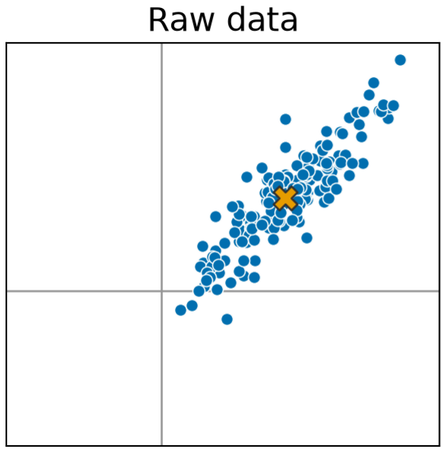
PCA in NumPy

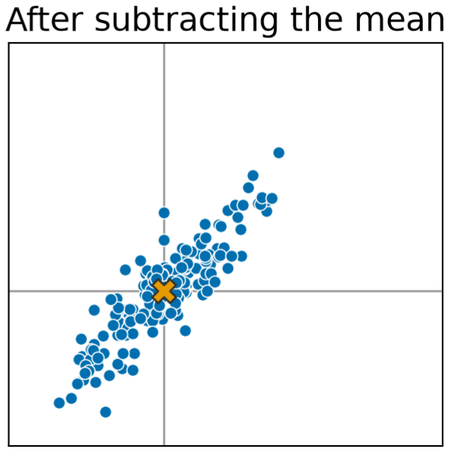
```
X = Xs                                          # (3000, 784), one image per row
mean = X.mean(axis=0)
_, _, Vt = np.linalg.svd(X - mean, full_matrices=False)   # rows of Vt: the components
Z = (X - mean) @ Vt[:2].T                       # keep the first 2
print(X.shape, "->", Z.shape)
```

```
(3000, 784) -> (3000, 2)
```

- The SVD of the centered data gives the same components as the eigenvectors of C, without ever building the 784×784 matrix
- Each row of `Vt` is one component: 784 weights
- `Z` is what we plotted: every image reduced to 2 dimensions

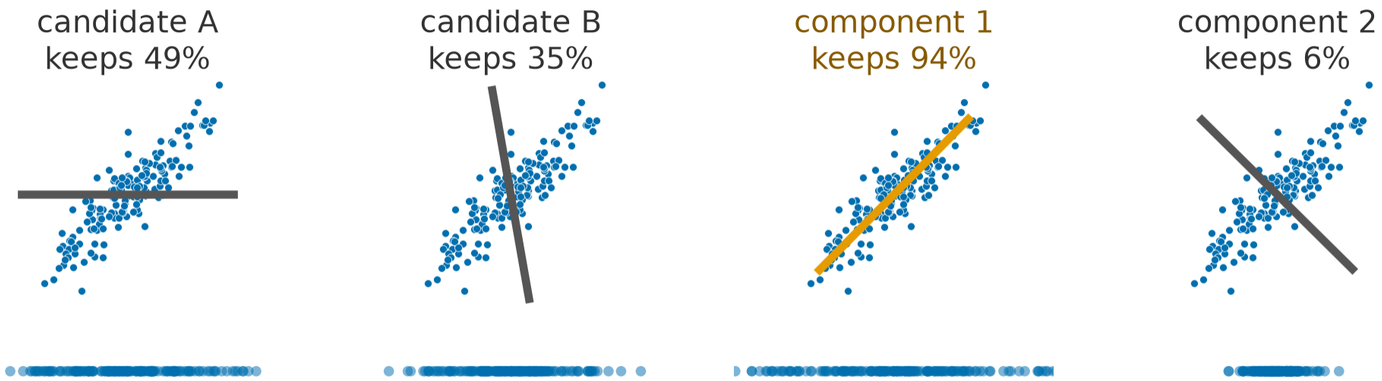
Breakdown: center the data





- We care about how the data varies around its center, so measure from it. Subtract the mean (orange X) from every point
- For images: subtract the average digit, which is why component weights were drawn as differences from it

Breakdown: find the components



- Try candidate dimensions $\mathbf{w}$. Each point gets a value $z = \mathbf{w}^\top \mathbf{x}$ (bottom row). More variance means more of the differences kept
- Component 1 has the most variance. Component 2 is the best one **perpendicular** to it, and so on
- The best candidates are the eigenvectors of C. Each eigenvalue is the variance along its component

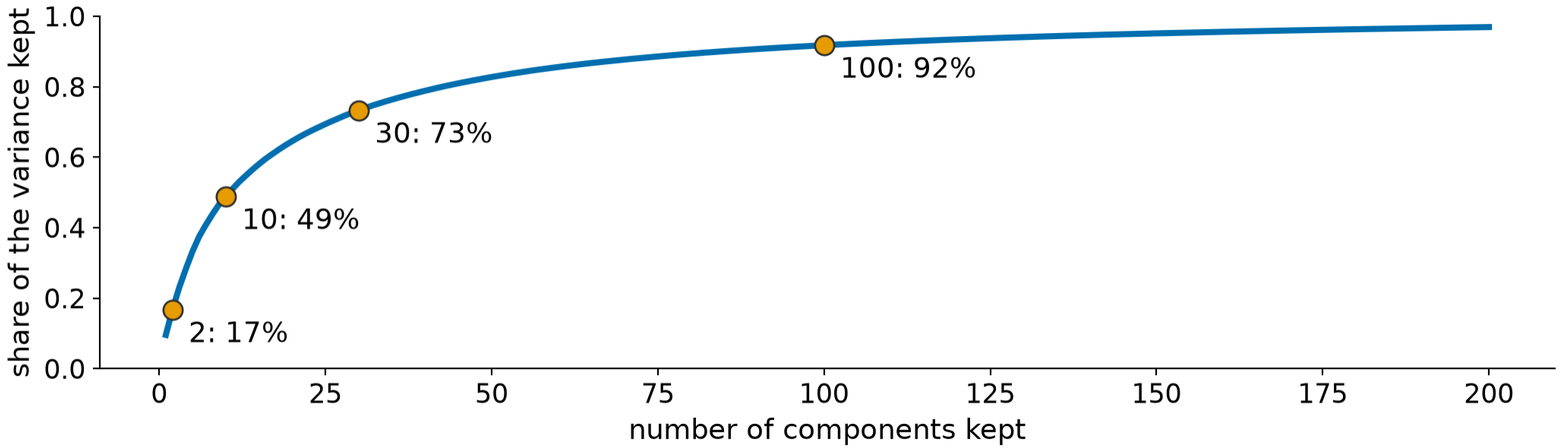
Breakdown: project and keep the top k

$$\mathbf{z} = W^\top(\mathbf{x} - \mu)$$

Keep k	$W^\top$	$\mathbf{x} - \mu$	$\mathbf{z}$
2	2×784	784	2
30	30×784	784	30
100	100×784	784	100

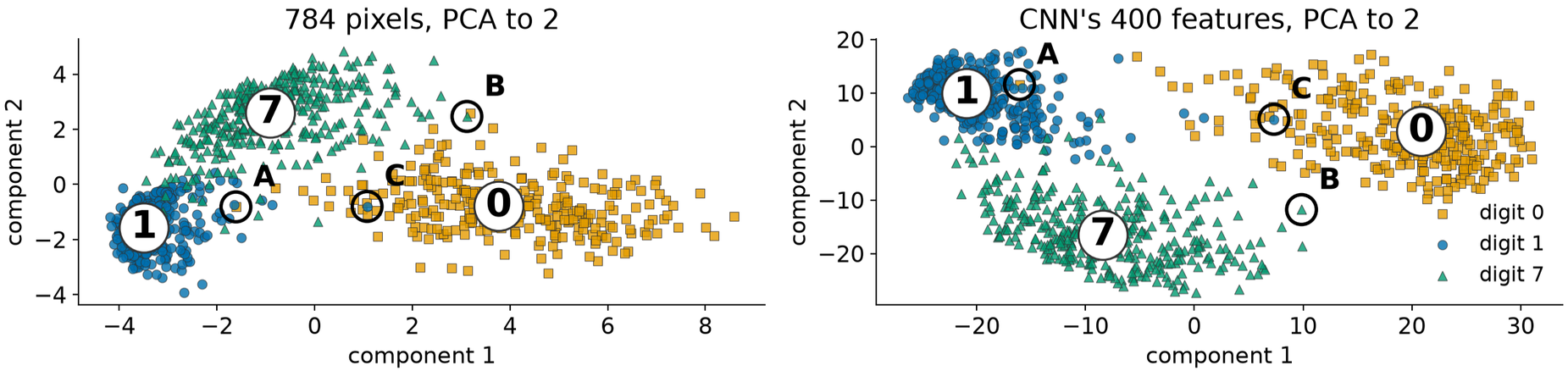
- Each row of $W^\top$ is one component, so each entry of $\mathbf{z}$ is one dot product
- The eigenvalues say how much each component keeps, so the **first** k keep the most variance of any k perpendicular dimensions

How many components do we need?



- On these 3,000 images, 2 components keep 17% of the variance, 30 keep 73%, and 100 keep 92%
- That matches the reconstructions: a blur at 2, a clear 7 at 30, close to the original at 100

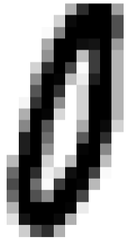
Applying PCA to a CNN’s features



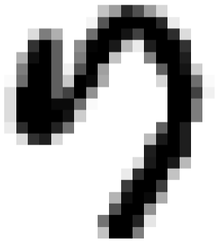
- Same 3,000 images, same tool. Left: the 784 pixels. Right: a CNN’s 400 features. This CNN was trained on **digits 0 to 6 only**, so it never saw a 7
- The CNN’s picture separates the three groups better, even the 7s it never saw. The pixel picture has 7s overlapping both
- Better, but PCA never sees labels: it keeps the biggest differences, not the separating ones

The odd ones out

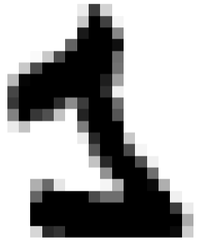
A: labeled 0



B: labeled 7

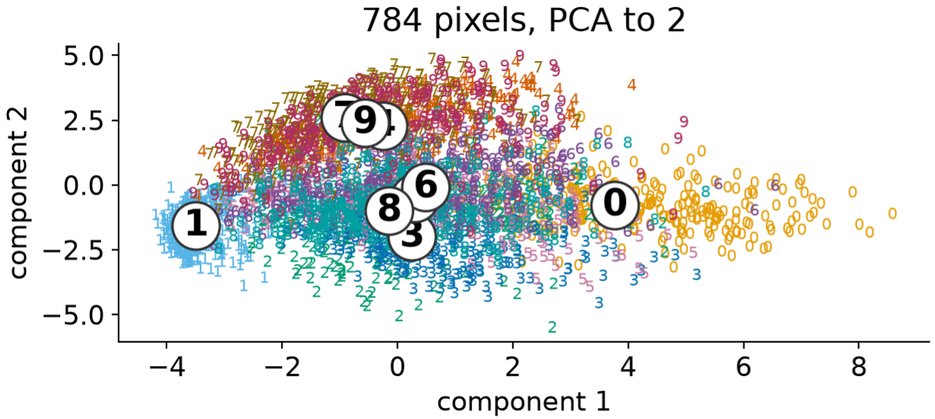


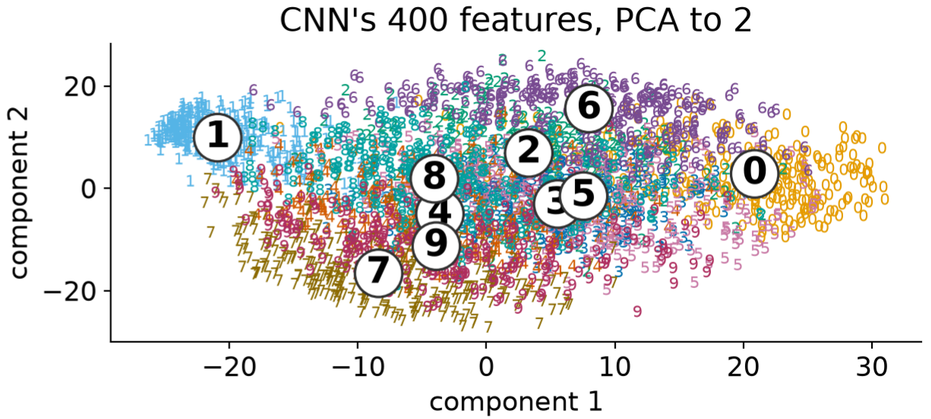
C: labeled 1



- **A** is a tall, thin 0. By its pixels and by the CNN's features, it lands next to the 1s
- **B**, an unusual 7, and **C**, a 1 with a heavy base, land among the 0s in the pixel picture. C has 2.2 times the ink of a typical 1
- Distances between pixels reward the same amount of ink in the same places, not the same digit

All ten digits





	% @ 2 components	Components for 50%	Components for 90%
PCA on 784 pixels	17%	11	85
PCA on CNN’s 400 features	28%	6	42

Takeaway

PCA builds new dimensions, each a linear combination of the original ones, that keep as much of the variance as possible.

It keeps the biggest differences, not necessarily the ones we care about.

Reusing a representation

Keep the features, replace the head

The Big Idea

A network trained on one task has learned a representation. For a new task with little data, keep that representation and train only a small new part on top.

The setup: a model trained on only some of the digits

What we already have

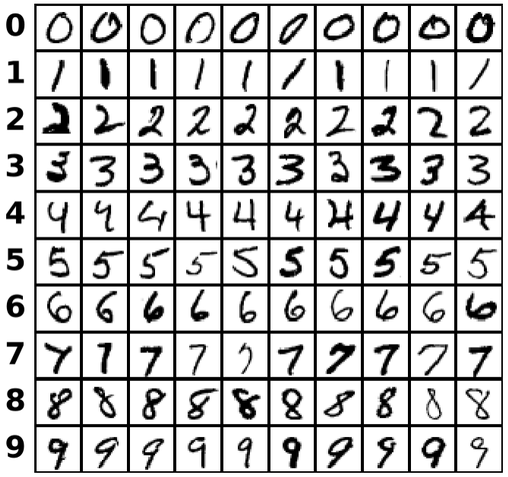
- The CNN from the PCA slides, trained on digits **0 to 6 only**: 41,935 images
- It has **never seen a 7, 8 or 9**, and is right 0% of the time on them

What we want

- A classifier for **all ten digits**
- But only **10 labeled examples of each**: 100 images in all

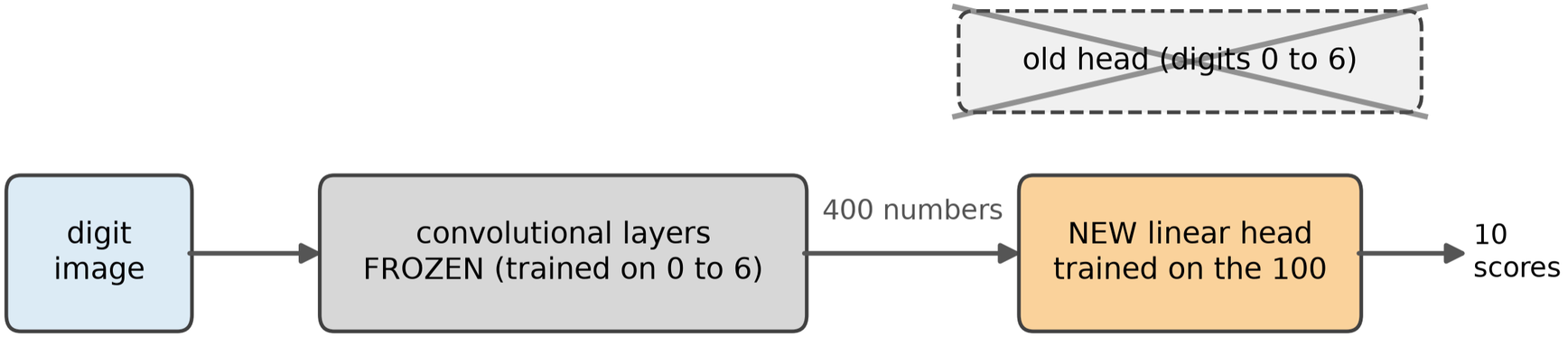
Can we reuse the CNN, instead of starting over with 100 images?

The new task's 100 labeled images



- 10 labeled examples of each digit, 100 images in all. This is everything we get to train the new classifier with
- Far too few to train a network from scratch with much confidence

Freeze the features, train a new head



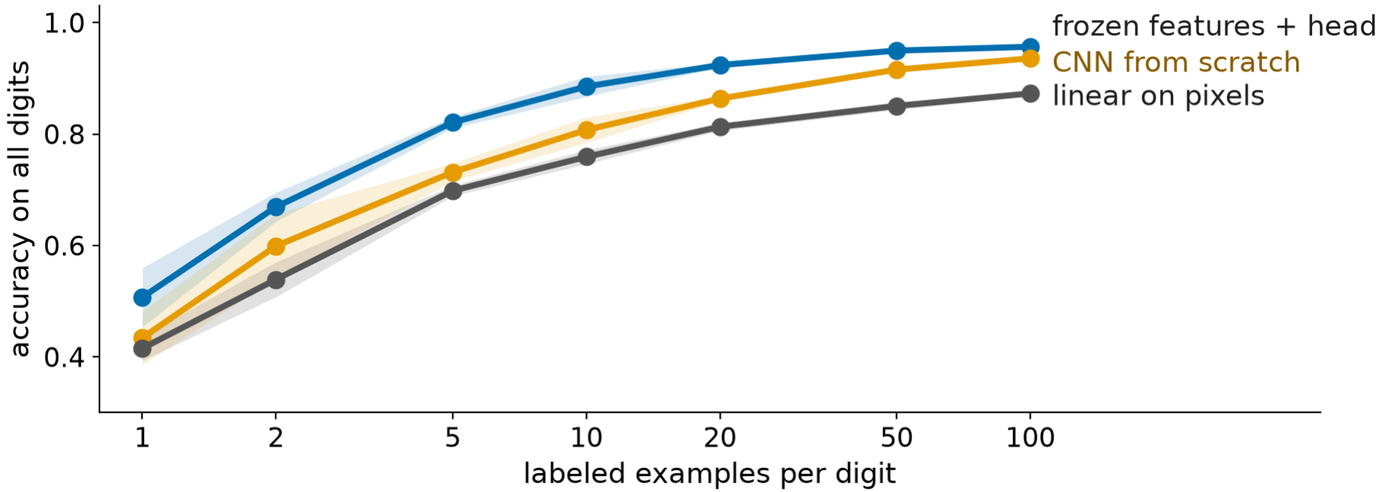
- The convolutional layers keep their weights, and the 400 numbers they output become the input to a new linear layer
- Only that linear layer is trained, so there are 4,010 weights to learn, not the whole network

How does it do?

Method	Trained on	Tested on all	on 0 to 6	on 7 to 9
Linear on pixels	the 100	76%	78%	70%
CNN from scratch	the 100	81%	82%	76%
Frozen features + head	0 to 6, then the 100	89%	91%	84%

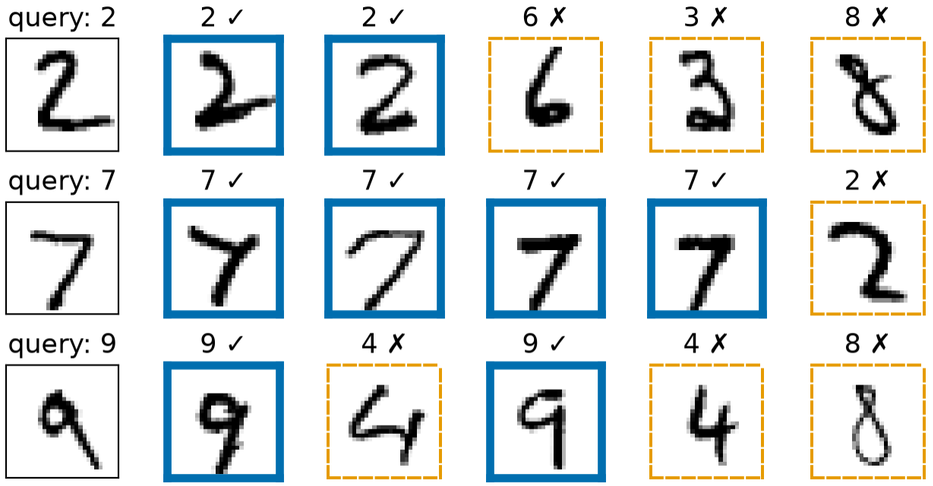
- “The 100” are the labeled images from the last slides, with 10 examples of **every** digit, 7, 8 and 9 included. Only the frozen CNN also learned from the 41,935 images of 0 to 6
- The last three columns say which digits the **test** images show. Averaged over the 10,000 validation images and 5 draws of the 100

How many labels do we need?



- Frozen features are ahead at every size. The lead over training from scratch is about 7 points at 1 per digit, and shrinks to 2 points at 100
- More labels help everyone, and the gap closes as they grow. The shaded bands are one standard deviation over 5 draws

Another way to reuse the features: KNN



- Instead of training a new head: find the labeled images whose features are closest, and copy their label
- Nothing to train, and a new class only needs its labeled images added to the pile
- Using the single nearest labeled image: 85% correct, against 89% for the new head and 75% for neighbors by pixels

Twin networks

Learn what “the same” means

Same or different?



- Sometimes we cannot list the classes ahead of time: new people, new products, new digits
- We can still ask about **two images**: are they the same kind of thing?
- It is the question from the start of the lecture: which two are the same digit?

The Big Idea

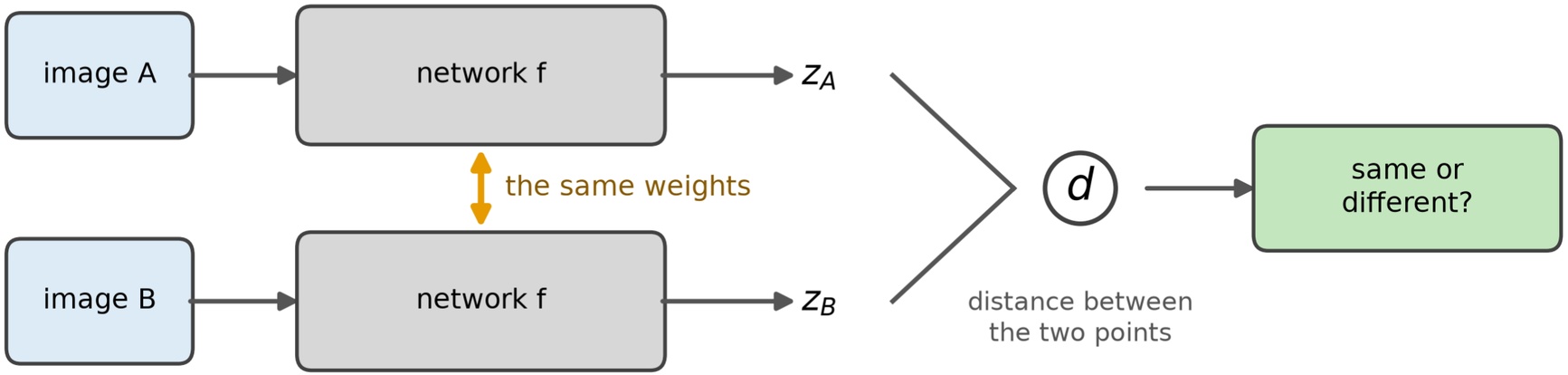
Instead of learning to name the classes, learn an *embedding* where things that are the same are close together and things that are different are far apart.

What is an embedding?

An embedding is a representation trained so that closeness between vectors says how similar the things are.

- Close (small distance, high similarity) means similar. Far means different
- Not every representation is one: pixels are a vector, but nearby pixel vectors need not be the same digit
- The vector a twin network gives an image is that image's embedding

One network, applied twice



- **One** network f is applied to both images. Same weights, as with a convolution applied at every position
- It maps an image to a point, here with 2 dimensions. The distance d between the two points is the answer

The twin network in PyTorch

```
 1 import torch
 2 from torch import nn
 3
 4 class Twin(nn.Module):
 5     def __init__(self, dim=2):
 6         super().__init__()
 7         self.f = nn.Sequential(                                  # the classifier's conv layers
 8             nn.Conv2d(1, 6, 3), nn.ReLU(), nn.MaxPool2d(2),
 9             nn.Conv2d(6, 16, 3), nn.ReLU(), nn.MaxPool2d(2),
10             nn.Flatten(), nn.Linear(400, dim))                   # down to `dim` numbers
11
12     def forward(self, x1, x2):
13         z1, z2 = self.f(x1), self.f(x2)                          # called twice
14         return (z1 - z2).pow(2).sum(1).add(1e-8).sqrt()          # distance per pair
15
16 dist = Twin()(torch.zeros(5, 1, 28, 28), torch.zeros(5, 1, 28, 28))
```

```
torch.Size([5])
```

- `self.f` is created once, so both calls use the same weights
- The small `1e-8` keeps the gradient finite when two points land on top of each other

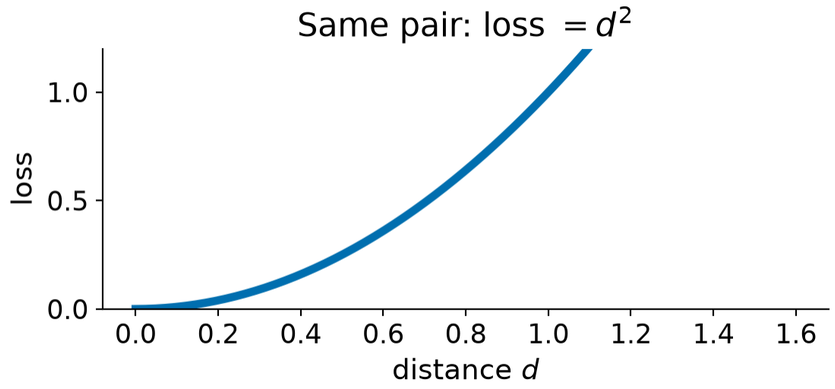
The contrastive loss

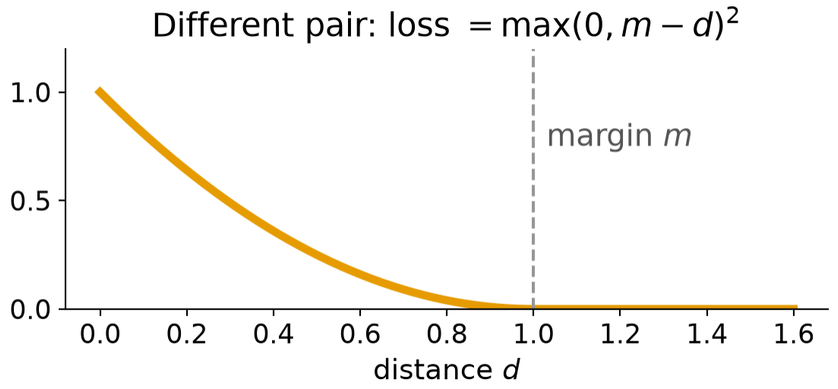
$$L = y\,d^2 + (1 - y)\,\max(0,\, m - d)^2$$

Pair	y	$1 - y$	The loss becomes
Same	1	0	$1 \cdot d^2 + 0 \cdot \max(0, m - d)^2 = d^2$
Different	0	1	$0 \cdot d^2 + 1 \cdot \max(0, m - d)^2 = \max(0, m - d)^2$

- y is the label we made for the pair, so it is always exactly 0 or 1. That makes y and $1 - y$ **switches**: for any one pair, one term is multiplied by 1 and the other by 0
- d is the distance between the two points, and m is the **margin**, which we set to 1. The loss for a batch is the average over its pairs

Breakdown: pull and push





Pull $(y = 1)$: the loss is d^2

- The further apart a same pair is, the larger the loss, so training moves the two points together
- At $d = 0$ the loss is 0: nothing left to fix

Push $(y = 0)$: the loss is $\max(0, m - d)^2$

- Closer than the margin $(d < m)$: $m - d$ is positive, so it costs something. The closer, the more
- At or past the margin $(d \geq m)$: $m - d$ is 0 or negative, the $\max$ gives 0, and there is no cost

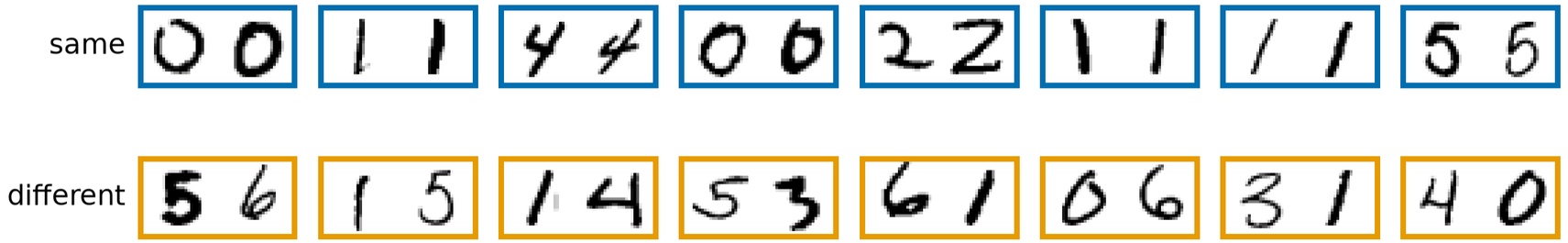
The loss with real numbers

Same pairs, $y = 1$	The formula with the numbers in	Loss
$d = 0.2$	$1 \cdot 0.2^2 + 0 \cdot \max(0,\, 1 - 0.2)^2$	0.04
$d = 0.9$	$1 \cdot 0.9^2 + 0 \cdot \max(0,\, 1 - 0.9)^2$	0.81

Different pairs, $y = 0$	The formula with the numbers in	Loss
$d = 0.3$	$0 \cdot 0.3^2 + 1 \cdot \max(0,\, 1 - 0.3)^2$	0.49
$d = 1.4$	$0 \cdot 1.4^2 + 1 \cdot \max(0,\, 1 - 1.4)^2$	0.00

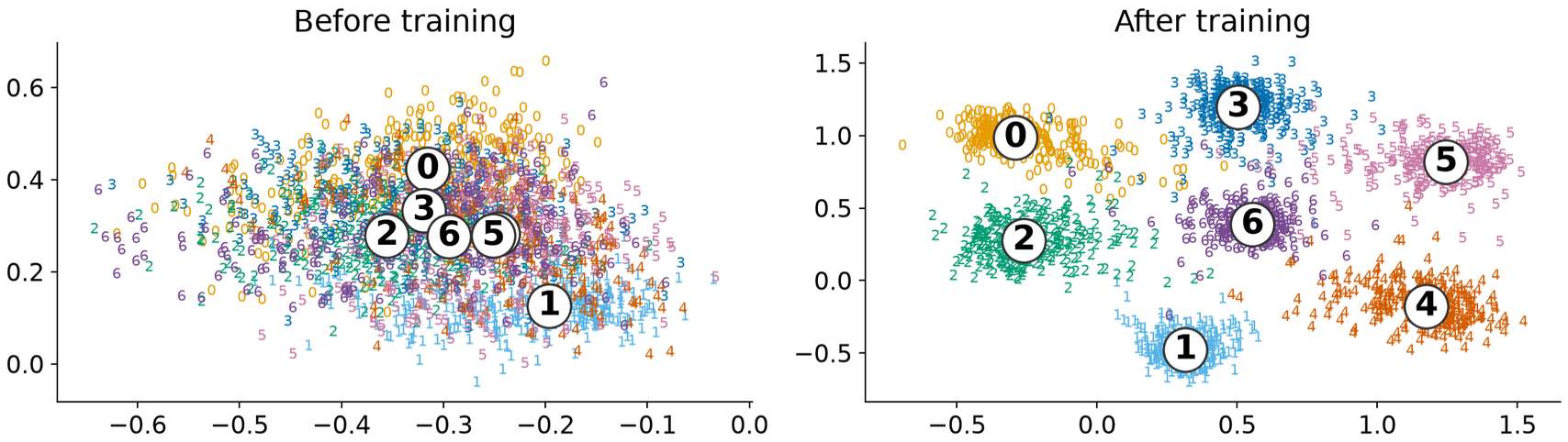
- Margin $m = 1$. One term is always multiplied by 0, so only the other one counts
- The same pair at 0.9 is pulled hard. The different pair at 0.3 is pushed. At 1.4 it is past the margin, so $\max(0, -0.4) = 0$ and nothing happens

Making the training data: pairs



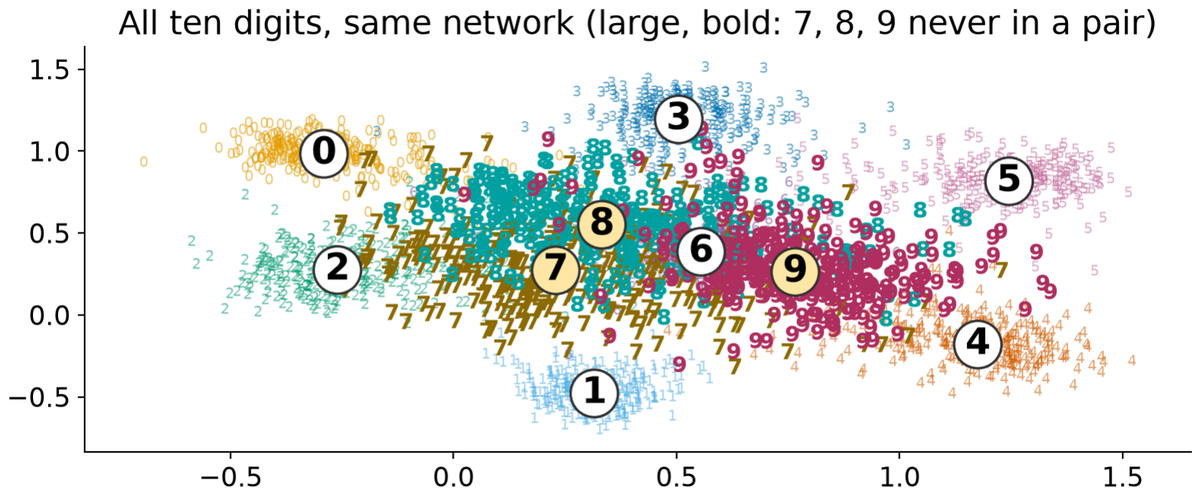
- Training data for a twin network is **pairs of images, each with a label**: same ($y = 1$) or different ($y = 0$)
- We make them from a labeled dataset: pick two images, and call it a same pair if their digits match
- Half the pairs of each kind. In this case the pairs use only digits **0 to 6**. A new batch of pairs is drawn every step

Before and after training



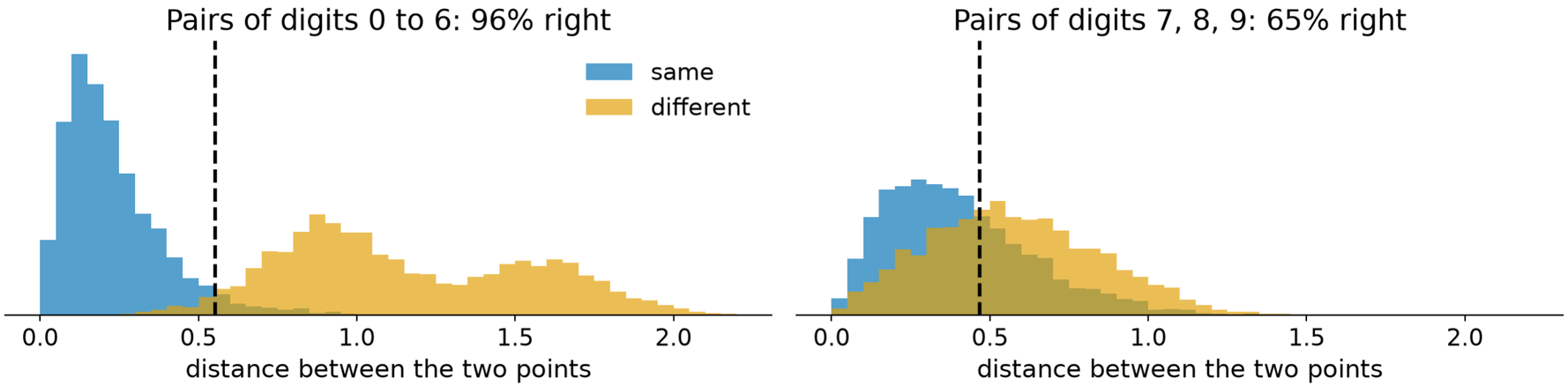
- Digits 0 to 6, from images that were never in a training pair. Each image is one point: the 2-D embedding itself, with no projection
- Before: one blob. After: a group for each digit

Digits it never saw



- 7, 8 and 9 were never in a training pair. The same network still places them
- The digits it knows stay in separate groups. 7, 8 and 9 land between them, on top of each other

Same or different, as a distance



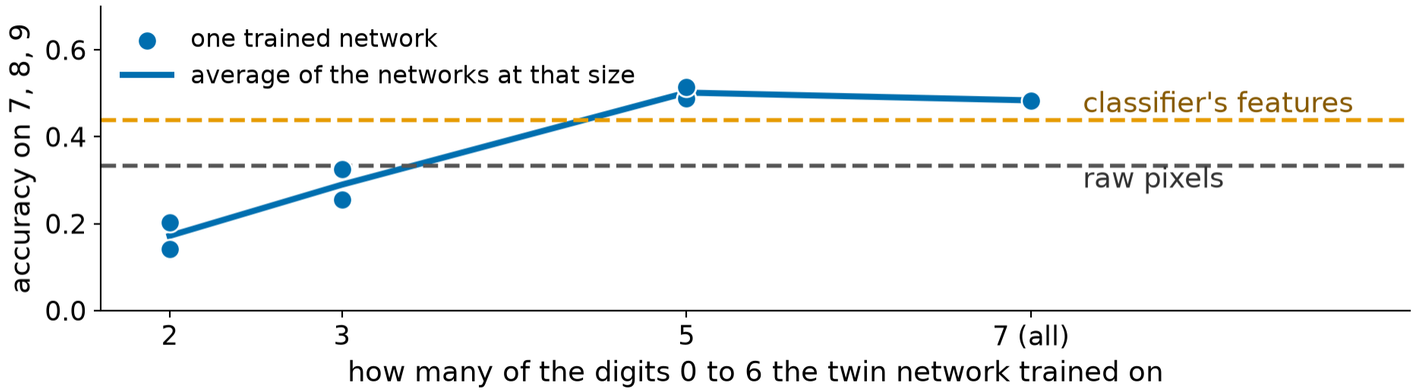
- Dashed line: the distance threshold that gets the most pairs right. Same pair if closer, different pair if further
- Digits it trained on: 96%, against 80% for the classifier's features and 68% for pixels. Digits 7, 8, 9: 65%, against 70% and 64%

Classifying with one example per digit

Closest in	All digits	Digits 0 to 6	Digits 7 to 9
Raw pixels	43%	46%	33%
Classifier's features	57%	62%	44%
Twin network, 2-D	67%	84%	25%
Twin network, 8-D	80%	93%	48%

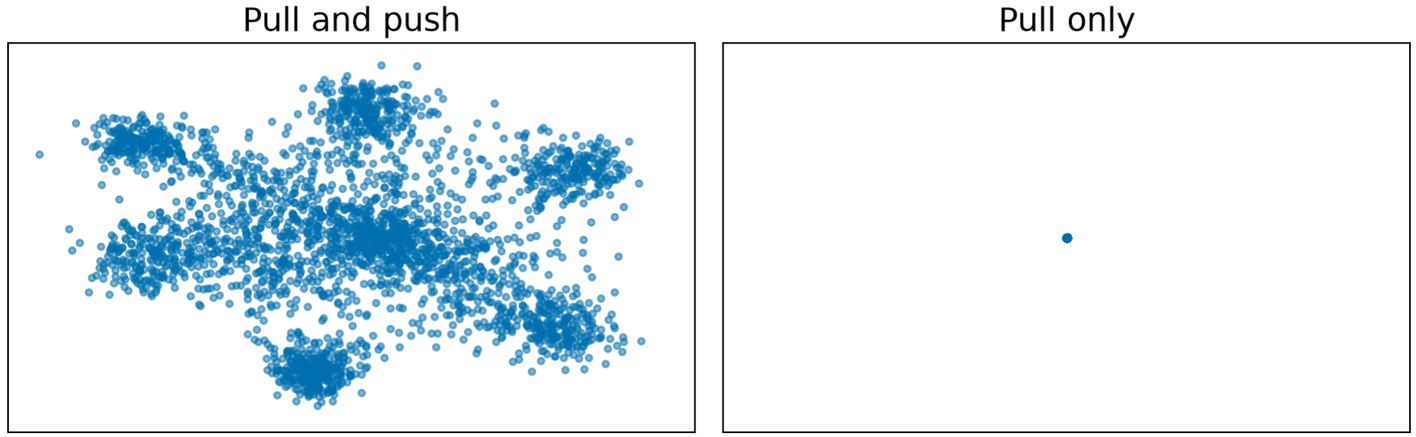
- One labeled image of each digit, ten in all, 7, 8 and 9 included. Each new image gets the label of the closest one. Entries: share labeled correctly
- Digits 7 to 9 are ones the twin never saw, and there it only just beats the classifier's features
- "Closest" is cosine similarity for pixels and features, and straight-line distance for the twin, which is how it was trained

Does it help to train on more classes?



- Does training on more different classes help with classes it never saw? Each network is 8-D, scored like the last slide, on 7, 8, 9
- For 2, 3 and 5 classes we trained two networks, each on a different random choice of digits. With all 7 there is only one
- Up to about 5 classes it helps. With 3 or fewer it does no better than raw pixels

Why we need the push term



- With only the pull term, the cheapest way to make same pairs close is to put **everything** in one place. The standard deviation falls from 0.50 to 0.0003
- We removed the push on purpose, to show why it is there. With both terms and a margin, as in everything before this slide, it does not happen

Takeaway

A classifier learns names for the classes it was given. A twin network learns what "the same" means, and that carries over to new classes only as far as the classes it trained on cover them.

Summary

Recap: three representations

	PCA	Classifier's features	Twin embedding
Built by	Keeping the most variance	Training to name the classes	Training on same / different pairs
Uses labels?	No	Yes	Yes, to make the pairs
Linear?	Yes	No	No
Good for	Looking at data, using fewer dimensions	Reusing on a new task with few labels	Asking whether two things are the same

Recap: key ideas

- A **representation** is a vector that describes an input, and the same input can be described in many ways
- **PCA** builds new dimensions, each a linear combination of the originals, that keep the most variance. It uses no labels
- A trained network's features can be **reused**: freeze them and train a new head on a few labeled examples, or compare with nearest neighbors
- A **twin network** trains the representation for the comparison itself: one network applied to both images, with a loss that pulls same pairs together and pushes different pairs apart, up to a margin
- How well that carries over to **new classes** depends on how many different classes it learned from

Where embeddings come back

- Text works the same way: words and tokens become vectors, and the goal is again that close means similar
- Modern contrastive methods, such as CLIP, are twin networks where the two sides are different networks, one for images and one for text
- Not covered here: fine-tuning and error analysis. Those come later, along with how to tell when reusing a network is a good idea